

3 Objects

Figure 3.1 credit: modification of work "Port of Melbourne", by Chris Phutully/Flickr, CC BY 2.0

Chapter Outline

3.1 Strings revisited
3.2 Formatted strings
3.3 Variables revisited
3.4 List basics
3.5 Tuple basics
3.6 Chapter summary

Introduction

An **object** is a single unit of data in a Python program. So far, this book has introduced three types of objects: strings, integers, and floats. This chapter takes a closer look at how strings are represented and how integers and floats can be formatted. To better understand what an object actually is, the relationship between variables and objects is emphasized. The chapter also introduces two types of containers: lists and tuples. A **container** is an object that can hold an arbitrary number of other objects. At the end of this chapter, you will be able to solve more complex problems using fewer variables.

3.1 Strings revisited

Learning objectives

By the end of this section you should be able to

- Extract a specific character from a string using an index.
- Use escape sequences to represent special characters.

Indexes

A string is a sequence of zero or more characters. Each character has an index that refers to the character's position. Indexes are numbered from left to right, starting at 0. Indexes are also numbered from right to left,

starting at -1.

CHECKPOINT

String indexes

String indexes; ch 3, video 1 (https://www.openstax.org/r/string-indexes)

CONCEPTS IN PRACTICE

String indexes

1. What is the index of the second character in a string?
   a. 1
   b. 2
   c. -2
2. If `s = "Python!"`, what is the value of `s[1] + s[-1]`?
   a. `"P!"`
   b. `"y!"`
   c. `"yn"`
3. If `s = "Python!"`, what type of object is `s[0]`?
   a. character
   b. integer
   c. string

Unicode

Python uses **Unicode**, the international standard for representing text on computers. Unicode defines a unique number, called a **code point**, for each possible character. Ex: `"P"` has the code point 80, and `"!"` has the code point 33.

The built-in `ord()` function converts a character to a code point. Ex: `ord("P")` returns the integer 80. Similarly, the built-in `chr()` function converts a code point to a character. Ex: `chr(33)` returns the string `"!"`.

Unicode is an extension of **ASCII**, the American Standard Code for Information Interchange. Originally, ASCII defined only 128 code points, enough to support the English language. Unicode defines over one million code

points and supports most of the world's written languages.

32 (space)	64 @	96 `
33 !	65 A	97 a
34 "	66 B	98 b
35 #	67 C	99 c
36 $	68 D	100 d
37 %	69 E	101 e
38 &	70 F	102 f
39 '	71 G	103 g
40 (	72 H	104 h
41)	73 I	105 i
42 *	74 J	106 j
43 +	75 K	107 k
44 ,	76 L	108 l
45 -	77 M	109 m
46 .	78 N	110 n
47 /	79 O	111 o
48 0	80 P	112 p
49 1	81 Q	113 q
50 2	82 R	114 r
51 3	83 S	115 s
52 4	84 T	116 t
53 5	85 U	117 u
54 6	86 V	118 v
55 7	87 W	119 w
56 8	88 X	120 x
57 9	89 Y	121 y
58 :	90 Z	122 z
59 ;	91 [	123 {
60 <	92 \	124 |
61 =	93]	125 }
62 >	94 ^	126 ~
63 ?	95 _	127 (delete)

Table 3.1 Character values. This table shows code points 32 to 127 as defined by ASCII and Unicode. Code points 0 to 31 are non-printable characters that were used for telecommunications.

CONCEPTS IN PRACTICE

ord() and chr()

4. What is the code point for the letter A?

a. 1

b. 65

c. 97

5. What value does `ord("0")` return?
   a. `0`
   b. `48`
   c. `Error`
6. What does `chr(126)` return?
   a. `~`
   b. `"~"`
   c. `Error`

Special characters

An **escape sequence** uses a backslash (\) to represent a special character within a string.

Escape sequence	Meaning	Example	Screen output
`\n`	A newline character that indicates the end of a line of text.	`print("Escape\nsequence!")`	`Escape` `sequence!`
`\t`	A tab character; useful for indenting paragraphs or aligning text on multiple lines.	`print("Escape\tsequence!")`	`Escape  sequence!`
`\'`	A single quote; an alternative to enclosing the string in double quotes.	`print('I\'ll try my best!')`	`I'll try my best`
`\"`	A double quote; an alternative to enclosing the string in single quotes.	`print("I heard you said \"Yes\"")`	`I heard you said "Yes"`
`\\`	A backslash character.	`print("This prints a \\")`	`This prints a \`

Table 3.2 Common escape sequences.

CONCEPTS IN PRACTICE

Tabs and newlines

7. Which of the following is an escape sequence?
   a. `t`

b. `/t`
c. `\t`

8. Which statement prints a backslash (\) to the screen?
   a. `print(\\)`
   b. `print(\"\")`
   c. `print("\\")`
9. Which statement prints `Enter` and `here` on separate lines?
   a. `print("Enter here")`
   b. `print("Enter" + \n + "here")`
   c. `print("Enter" + "\n" + "here")`

TRY IT

Hopper quote

Grace Hopper (https://openstax.org/r/100gracehopper) (1906–1992) was a famous computer scientist (and rear admiral in the US Navy!) who came up with the idea of machine-independent programming languages. She envisioned a programming language based on English and made many contributions that paved the way for modern programming languages, including Python.

Write a program that prints the following text, including the quotation marks. Your program may not use single quotes (') anywhere in the code. The last line must be indented with a tab character.

```
"To me programming is more than an important practical art.
It is also a gigantic undertaking in the foundations of knowledge."
-- Grace Hopper
```

Access multimedia content (http://openstax.org/books/introduction-python-programming/pages/3-1-strings-revisited)

TRY IT

Shift cipher

During the Roman Empire, Julius Caesar (100–44 BCE) used a simple technique to encrypt private messages. Each letter of the message was replaced with the third next letter of the alphabet. Ex: If the message was CAT, the C became F, the A became D, and the T became W, resulting in the message FDW. This technique is known as a shift cipher because each letter is shifted by some amount. In Caesar's case, the amount was three, but other amounts (besides 0) would work too.

Write a program that prompts the user to input the following two values (example input in bold):

```
Enter a 3-letter word: CAT
Shift by how many letters? 3
```

The program should then shift each letter of the word by the desired amount. Based on the example above, the output would be:

```
The secret message is: FDW
```

Hint: Use the `ord()` function to convert each letter to an integer, add the shift amount to each integer, use the `chr()` function to convert each integer to a character, and concatenate the resulting characters.

Access multimedia content (http://openstax.org/books/introduction-python-programming/pages/3-1-strings-revisited)

3.2 Formatted strings

Learning objectives

By the end of this section you should be able to

- Use f-strings to simplify output with multiple values.
- Format numbers with leading zeros and fixed precision.

F-strings

A **formatted string literal** (or **f-string**) is a string literal that is prefixed with `"f"` or `"F"`. A **replacement field** is an expression in curly braces ({}) inside an f-string. Ex: The string `f"Good morning, {first} {last}!"` has two replacement fields: one for a first name, and one for a last name. F-strings provide a convenient way to combine multiple values into one string.

CHECKPOINT

Printing an f-string

Printing an f-string; ch 3, video 2 (https://www.openstax.org/r/printing-f-string)

CONCEPTS IN PRACTICE

Basic f-strings

1. What is the output of the following code?

```
animal = "dog"
says = "bark"
print(f"My {animal} says {says} {says} {says}")
```

   a. `My dog bark bark bark bark`
   b. `My dog says bark bark bark`
   c. `Error`

2. What is the output of the following code?

```
temp = "hot"
```

```
food = "potato"
print("{temp} {food}")
```

a. {temp} {food}
b. hot potato
c. Error

3. How can the following code be rewritten using an f-string?

```
print(x, "+", y, "=", x + y)
```

a. `print(f"x + y = {x+y}")`
b. `print(f"{x + y} = {x + y}")`
c. `print(f"{x} + {y} = {x+y}")`

Formatting numbers

Programs often need to display numbers in a specific format. Ex: When displaying the time, minutes are formatted as two-digit integers. If the hour is 9 and the minute is 5, then the time is "9:05" (not "9:5").

In an f-string, a replacement field may include a format specifier introduced by a colon. A **format specifier** defines how a value should be formatted for display. Ex: In the string `f"{hour}:{minute:02d}"`, the format specifier for `minute` is `02d`.

Format	Description	Example	Result
`d`	Decimal integer (default integer format).	`f"{12345678:d}"`	`'12345678'`
`,d`	Decimal integer, with comma separators.	`f"{12345678:,d}"`	`'12,345,678'`
`10d`	Decimal integer, at least 10 characters wide.	`f"{12345678:10d}"`	`'  12345678'`
`010d`	Decimal integer, at least 10 characters wide, with leading zeros.	`f"{12345678:010d}"`	`'0012345678'`
`f`	Fixed-point (default is 6 decimal places).	`f"{math.pi:f}"`	`'3.141593'`
`.4f`	Fixed-point, rounded to 4 decimal places.	`f"{math.pi:.4f}"`	`'3.1416'`
`8.4f`	Fixed-point, rounded to 4 decimal places, at least 8 characters wide.	`f"{math.pi:8.4f}"`	`'  3.1416'`
`08.4f`	Fixed-point, rounded to 4 decimal places, at least 8 characters wide, with leading zeros.	`f"{math.pi:08.4f}"`	`'003.1416'`

Table 3.3 Example format specifiers. The table shows common ways that numbers can be formatted. Many more formatting options are available and described in Python's Format Specification Mini-Language (https://docs.python.org/3/library/string.html#formatspec).

CONCEPTS IN PRACTICE

Formatting numbers

4. What f-string formats a date as MM/DD/YYYY?
   a. `f"{month:02d}/{day:02d}/{year}"`
   b. `f"{month:2d}/{day:2d}/{year}"`
   c. `f"{month}/{day}/{year}"`
5. What statement displays the variable `money` rounded to two decimal places?
   a. `print(f"{money:2d}")`
   b. `print(f"{money:2f}")`
   c. `print(f"{money:.2f}")`
6. What format specifier displays a floating-point number with comma separators and rounded to two decimal places?
   a. `.2,f`
   b. `,.2f`
   c. `,d.2f`

TRY IT

Mad lib (f-string)

A mad lib (https://openstax.org/r/100fstring) is a funny story that uses words provided by a user. The following mad lib is based on four words (user input in bold):

```
Enter a name: Buster
Enter a noun: dog
Enter an adjective: super
Verb ending in -ing: swimming

Buster, the super dog, likes to go swimming.
```

Most of the code for this mad lib is already written. Complete the code below by writing the f-string.

Access multimedia content (http://openstax.org/books/introduction-python-programming/pages/3-2-formatted-strings)

TRY IT

Wage calculator

You just landed a part-time job and would like to calculate how much money you will earn. Write a program that inputs the time you start working, the time you stop working, and your hourly pay rate (example input in bold):

```
Starting hour: 9
Starting minute: 30
Stopping hour: 11
Stopping minute: 0
Hourly rate: 15
```

Based on the user input, your program should calculate and display the following results:

```
Worked 9:30 to 11:00
Total hours: 1.5
Payment: $22.50
```

For this exercise, you need to write code that (1) calculates the total payment and (2) formats the three output lines. Use f-strings and format specifiers to display two-digit minutes, one decimal place for hours, and two decimal places for payment. The input code has been provided as a starting point.

Assume the use of a 24-hour clock. Ex: 16:15 is used instead of 4:15pm.

Access multimedia content (http://openstax.org/books/introduction-python-programming/pages/3-2-formatted-strings)

3.3 Variables revisited

Learning objectives

By the end of this section you should be able to

- Distinguish between variables, objects, and references.
- Draw memory diagrams with integers, floats, and strings.

References to objects

In Python, every variable refers to an object. The assignment statement `message = "Hello"` makes the variable `message` refer to the object `"Hello"`. Multiple variables may refer to the same object. Ex: `greeting = message` makes `greeting` refer to the same object as `message`. A **memory diagram** shows the relationship between variables and objects.

CHECKPOINT

Example memory diagram

Example memory diagram; ch 3, video 3 (https://www.openstax.org/r/memory-diagram)

CONCEPTS IN PRACTICE

Variables and objects

1. How many assignment statements are in the above animation?
   a. 2
   b. 3
   c. 4

2. Which of the following best describes the objects assigned?
   a. two float objects
   b. two int objects, one float object
   c. one int object, one float object
3. What symbol is used to show a variable's current value?
   a. an arrow
   b. a small black box
   c. a rounded box

EXPLORING FURTHER

Python Tutor (https://openstax.org/r/100pythontutor) is a free online tool for visualizing code execution. A user can enter any Python code, click Visualize Execution, and then click the Next button to run the code one line at a time. Here is the rating and score example (https://openstax.org/r/100ratingscore) from the animation above.

Python Tutor is also useful for drawing memory diagrams similar to the ones in this book. Before clicking Visualize Execution, change the middle option from "inline primitives, don't nest objects [default]" to "render all objects on the heap (Python/Java)" as shown in the following screenshot:

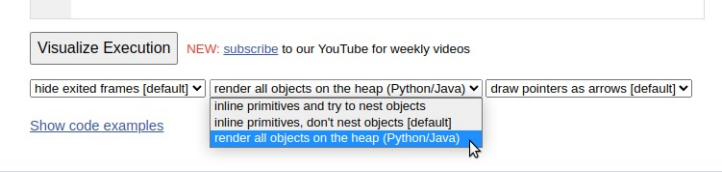


Figure 3.2

Properties of objects

Every object has an identity, a type, and a value:

- An object's **identity** is a unique integer associated with the object. Generally, this integer refers to the memory location where the object is stored. Once created, an object's identity never changes. The built-in `id()` function returns the object's identity.
- An object's **type** determines the possible values and operations of an object. Ex: Integers and floats can be "divided" using the `/` operator, but strings cannot. The built-in `type()` function returns the object's type.
- An object's **value** represents the current state of the object. Many objects, such as numbers and strings, cannot be modified once created. Some objects, such as lists (introduced later), are designed to be modified.

CHECKPOINT

Identity, type, and value

Identity, type, and value; ch 3, video 4 (https://www.openstax.org/r/identity-type-value)

CONCEPTS IN PRACTICE

id() and type()

4. Which value might be returned by `id(rating)`?
   a. `9793344`
   b. `<class 'float'>`
   c. `10.0`
5. Which value might be returned by `type(rating)`?
   a. `10.0`
   b. `"float"`
   c. `<class 'float'>`
6. What expression returns the value of an object?
   a. `value(rating)`
   b. `rating`
   c. `"rating"`

EXPLORING FURTHER

As shown in a memory diagram, variables and objects are two separate ideas. Calling a function like `id()` or `type()` returns information about an object, not a variable. In fact, a variable doesn't have an identity or a type, as shown in this example:

```
>>> rating = 10  # Integer object somewhere in memory.
>>> type(rating)
<class 'int'>
>>> id(rating)
9793344
>>> rating = "ten"  # String object somewhere else in memory.
>>> type(rating)
<class 'str'>
>>> id(rating)
140690967388272
```

One might incorrectly think that the rating variable's type or identity changes. However, the only thing that changes is which object the rating variable refers to.

TRY IT

Three variables

1. Draw a memory diagram for the following code:

```
a = 1
b = 2
```

```
c = b
b = a
a = c
```

2. Run the code on Python Tutor (https://openstax.org/r/100pythruncode) to check your answer.
3. Based on your diagram, answer these questions:
   - What is the final value of a, b, and c?
   - How many integer objects are created?

TRY IT

Different types

1. Draw a memory diagram for the following code:

```
name = "Chocolate"
length = len(name)
price = 1.99
lower = min(length, price)
product = name
name = name * 2
```

2. Run the code on Python Tutor (https://openstax.org/r/100pythruncode) to check your answer.
3. Based on your diagram, answer these questions:
   - What is the type and value of each object?
   - Which object does each variable reference?

3.4 List basics

Learning objectives

By the end of this section you should be able to

- Use indexes to access individual elements in a list.
- Use indexes to modify individual elements in a list.
- Use `len()` function to find the length of a list.
- Demonstrate that lists can be changed after creation.

Lists

A list object can be used to bundle elements together in Python. A list is defined by using square brackets `[]` with comma separated values within the square brackets. Ex: `list_1 = [1, 2, 4]`.

Empty lists can be defined in two ways:

- `list_1 = []`
- `list_1 = list()`

Lists can be made of elements of any type. Lists can contain integers, strings, floats, or any other type. Lists can also contain a combination of types. Ex: `[2, "Hello", 2.5]` is a valid list.

Python lists allow programmers to change the contents of the list in various ways.

CHECKPOINT

Lists

Lists: ch 3, video 5 (https://www.openstax.org/r/lists)

CONCEPTS IN PRACTICE

Lists

1. Which is the correct way to make a list of the numbers 3, 4, and 5?
   a. `new_list == [3, 4, 5]`
   b. `new_list = [3, 4, 5]`
   c. `new_list = (3, 4, 5)`
2. Which of the following is not a valid way to specify a list in Python?
   a. `my_list = [2, 2 3, 23]`
   b. `my_list = ["Jimmy", 2, "times"]`
   c. `my_list = ["C", "C++", "Python", "Java", "Rust", "Scala"]`

Using indexes

Individual list elements can be accessed directly using an index. Indexes begin at 0 and end at one less than the length of the sequence. Ex: For a sequence of 50 elements, the first position is 0, and the last position is 49.

The index number is put in square brackets `[]` and attached to the end of the name of the list to access the required element. Ex: `new_list[3]` accesses the 4th element in `new_list`. An expression that evaluates to an integer number can also be used as an index. Similar to strings, negative indexing can also be used to address individual elements. Ex: Index -1 refers to the last element and -2 the second-to-last element.

The `len()` function, when called on a list, returns the length of the list.

EXAMPLE 3.1

List indexes and len() function

The following code demonstrates the use of list indexes and the `len()` function. Line 6 shows the use of the `len()` function to get the length of the list. Line 10 shows how to access an element using an index. Line 14 shows how to modify a list element using an index.

```
1  # Setup a list of numbers
2  num_list = [2, 3, 5, 9, 11]
3  print(num_list)
4
5  # Print the length of the list
6  print("Length: ", len(num_list))
7
8  # Print the 4th element in the list
9  # The number 3 is used to refer to the 4th element
10 print("4th element:", num_list[3])
```

```
11
12 # The desired value of the 4th element is actually 7
13 # Update the value of the 4th element to 7
14 num_list[3] = 7
15
16 # The list of the first 5 prime numbers
17 print(num_list)
```

The above code's output is:

```
[2, 3, 5, 9, 11]
Length: 5
4th element: 9
[2, 3, 5, 7, 11]
```

CONCEPTS IN PRACTICE

List indexes and the len() function

3. Which is the correct way to access the 17th element in a list called `cardList`?
   a. `card_list[17]`
   b. `16)card_list(16)`
   c. `card_list[16]`

4. What is the correct way to modify the element `"Tom"` to `"Tim"` in the following list?

   ```
   name_list = ["Carla", "Monique", "Westin", "Tom"]
   ```

   a. `name_list[-1] = "Tim"`
   b. `name_list[4] = "Tim"`
   c. `name_list[end] = "Tim"`

TRY IT

List basics

Write a program to complete the following:

1. Create a list with the following elements: 2, 23, 39, 6, -5.
2. Change the third element of the list (index 2) to 35.
3. Print the resulting list and the list's length.

Access multimedia content (http://openstax.org/books/introduction-python-programming/pages/3-4-list-basics)

3.5 Tuple basics

Learning objectives

By the end of this section you should be able to

- Describe the features and benefits of a tuple.
- Develop a program that creates and uses a tuple successfully.
- Identify and discuss the mutability of a tuple.

Creating tuples and accessing elements

A **tuple** is a sequence of comma separated values that can contain elements of different types. A tuple must be created with commas between values, and conventionally the sequence is surrounded by parentheses. Each element is accessed by index, starting with the first element at index 0.

`tuple_1 = (2, 3, 4)`
`print(f'tuple_1: {tuple_1}')`
`print(tuple_1[1])`
`print()`
`data_13 = ('Aimee Perry', 96, [94, 100, 97, 93])`
`print(f'data_13: {data_13}')`
`print(data_13[2])`
	`tuple_1: (2, 3, 4)`
	`3`
	
	`data_13: ('Aimee Perry', 96, [94, 100, 97, 93])`
	`[94, 100, 97, 93]`

Table 3.4 Example tuples.

CONCEPTS IN PRACTICE

Creating tuples and accessing elements

1. Consider the example above. Which accesses `tuple_1`'s first element?
   a. `tuple_1[0]`
   b. `tuple_1[1]`
   c. `tuple_1[2]`
2. Which creates a valid tuple?
   a. `tuple_2 = (42, 26, 13)`
   b. `tuple_3 = (42, 26, 13.5)`
   c. both
3. Which creates a tuple with three elements?
   a. `my_tuple = 'a', 'b', 'c'`
   b. `my_tuple = ['a', 'b', 'c']`
   c. `my_tuple = ('a', 'b', 'c')`

Tuple properties

How do tuples compare to lists? Tuples are ordered and allow duplicates, like lists, but have different

mutability. An **immutable** object cannot be modified after creation. A **mutable** object can be modified after creation. Tuples are immutable, whereas lists are mutable.

CHECKPOINT

Mutability of a tuple vs. a list

Mutability of a tuple vs. a list; ch 3, video 6 (https://www.openstax.org/r/mutability-tuple-list)

CONCEPTS IN PRACTICE

Using tuples

4. Consider the final program in the example above. What is the value of `my_tuple`?
   a. `(0.693, 0.414, 3.142)`
   b. `(0.693, 1.414, 3.142)`
   c. `Error`

5. Why does the following code produce an error?

```
tuple_1 = ('alpha', 'bravo', 'charlie')
tuple_1.append('charlie')
```

   a. Append isn't allowed.
   b. Duplicates aren't allowed.
   c. Strings aren't allowed

6. A programmer wants to create a sequence of constants for easy reference that can't be changed anywhere in the program. Which sequence would be the most appropriate?
   a. list
   b. tuple

MUTABILITY AND PERFORMANCE

Tuples are immutable and have a fixed size, so tuples use less memory. Overall, tuples are faster to create and access, resulting in better performance that can be noticeable with large amounts of data.

TRY IT

Creating a tuple from a list

Suppose a programmer wants to create a tuple from a list to prevent future changes. The `tuple()` function creates a tuple from an object like a list. Ex: `my_tuple = tuple(my_list)` creates a tuple from the list `my_list`. Update the program below to create a tuple `final_grades` from the list grades.

Access multimedia content (http://openstax.org/books/introduction-python-programming/pages/3-5-tuple-basics)

TRY IT

Creating a tuple with user input

Write a program that reads in two strings and two integers from input and creates a tuple, `my_data`, with the four values.

Given input:

```
x
y
15
20
```

The output is:

```
my_data: ('x', 'y', 15, 20)
```

Access multimedia content (http://openstax.org/books/introduction-python-programming/pages/3-5-tuple-basics)

3.6 Chapter summary

Highlights from this chapter include:

- A **string** is a sequence of values that represents Unicode code points.
- An **index** refers to the position of a value in a sequence (string, list, tuple).
- Positive indexes range from 0 to length–1. Negative indexes range from –1 to –length.
- F-strings are a convenient way to print multiple outputs and format integers and floats.
- Variables refer to objects. Memory diagrams are useful for drawing variables and objects.
- A list object can be used to refer to multiple objects, by index, using the same variable.
- A tuple is similar to a list, but uses parentheses and cannot be changed once created.

Code	Description
`ord(c)`	Gets an integer representing the Unicode code point of a character.
`chr(i)`	Converts a Unicode code point (integer) into a One-character string.
`'\n'`	Escape sequence for the newline character.

Table 3.5 Chapter 3 reference.

Code	Description
`'\t'`	Escape sequence for the tab character.
`f"{number:.02d}"`	Creates a string by formatting an integer to be at least two digits.
`f"{number:.2f}"`	Creates a string by formatting a float to have two decimal places.
`id(object)`	Gets the identity (memory location) of an object.
`type(object)`	Gets the type (class name) of an object.
`my_list = ["a", "b", "c"]`	Creates a list of three strings.
`my_tuple = ("a", "b", "c")`	Creates a tuple of three strings.
`my_list[0]`	Gets the first element (`"a"`) of `my_tuple`.
`my_tuple[-1]`	Gets the last element (`"c"`) of `my_tuple`.

Table 3.5 Chapter 3 reference.